

Practical 4

Write a Python program that implements the page-replacement algorithm : FIFO, optimal

FIFO Code :

```
def pageFaults(pages, capacity):
    frames = []
    page_faults = 0
    page_hits = 0
    for page in pages:
        if page not in frames:
            page_faults += 1
            if len(frames) < capacity:
                frames.append(page)
            else:
                frames.pop(0)
                frames.append(page)
        else:
            page_hits += 1
    print("No. of hits =", page_hits)
    print("No. of misses =", page_faults)
    return page_faults
pages = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
capacity = 3
print("Page Faults (FIFO):", pageFaults(pages, capacity))
```

FIFO Output :

```
No. of hits = 3
No. of misses = 10
Page Faults (FIFO): 10
```

Optimum Code :

```
def search(key, frames):
    return key in frames
def predict(pages, frames, index):
    farthest = -1
    replace_index = -1
```

```
for i in range(len(frames)):
    try:
        next_use = pages.index(frames[i], index)    except
ValueError:
    return i
    if next_use > farthest:
        farthest = next_use
        replace_index = i
    return replace_index if replace_index != -1 else 0
def optimalPage(pages, capacity):
    frames = []
    hits = 0
    for i in range(len(pages)):
        page = pages[i]
        if search(page, frames):
            hits += 1
            continue
        if len(frames) < capacity:
            frames.append(page)
        else:
            replace_idx = predict(pages, frames, i + 1)
            frames[replace_idx] = page
    print("No. of hits =", hits)
    print("No. of misses =", len(pages) - hits)
pages = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
capacity = 3
optimalPage(pages, capacity)
```

Optimum Output :

```
No. of hits = 6
No. of misses = 7
```